

Predictive Modeling of Epileptic Seizure Detection Using Machine Learning

Rahul Rajput

2026-04-16

1. Introduction

The dataset that is used in this study is BEED of UCI repository, which includes EEG signal attributes (X1 - X16) and a target variable (y) that is a measure of seizure activity. The objective is to make predictive models that are capable of correctly classifying seizure and non-seizure cases to assist in clinical decision-making.

2. Methods and Analysis

The data set was initially purged of any missing data to allow proper analysis and the target variable was turned into a simple binary data (seizure vs non-seizure) to facilitate classification. This was followed by dividing the data into 80 percent training and 20 percent testing, which will enable the model to learn efficiently and also to evaluate the performance on the unseen data fairly. This study employed three models: Logistic Regression as an interpretable and simple baseline, Random Forest to learn the intricate trends in the data, and XGBoost to have a better prediction performance due to the use of advanced boosting models.

2.1 Libraries

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Loading required package: lattice
##
```

```
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
## lift
```

```
library(ranger)
```

```
## Warning: package 'ranger' was built under R version 4.5.3
```

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.5.3
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
## cov, smooth, var
```

```
library(knitr)
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.3
```

```
## here() starts at C:/Users/admin/Downloads
```

2.2 Load Data

```
file_path <- here::here("data", "BEED_Data.csv")

if (file.exists(file_path)) {
  df <- read.csv(file_path)
} else {
  stop("Data file not found. Ensure BEED_Data.csv is inside /data folder")
}
```

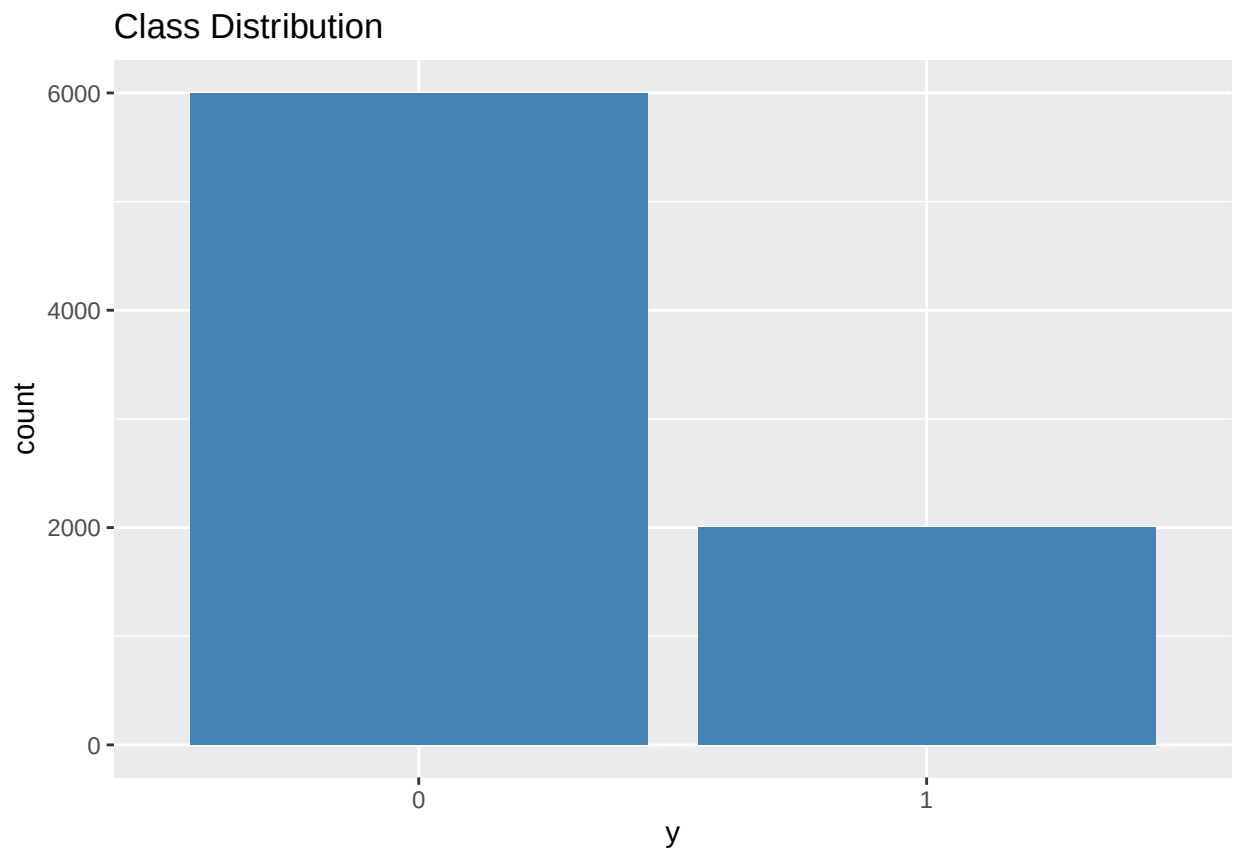
2.3 Preprocessing

```
df <- na.omit(df)
df$y <- ifelse(df$y == 1, 1, 0)
df$y <- as.factor(df$y)
```

2.4 Data Visualization

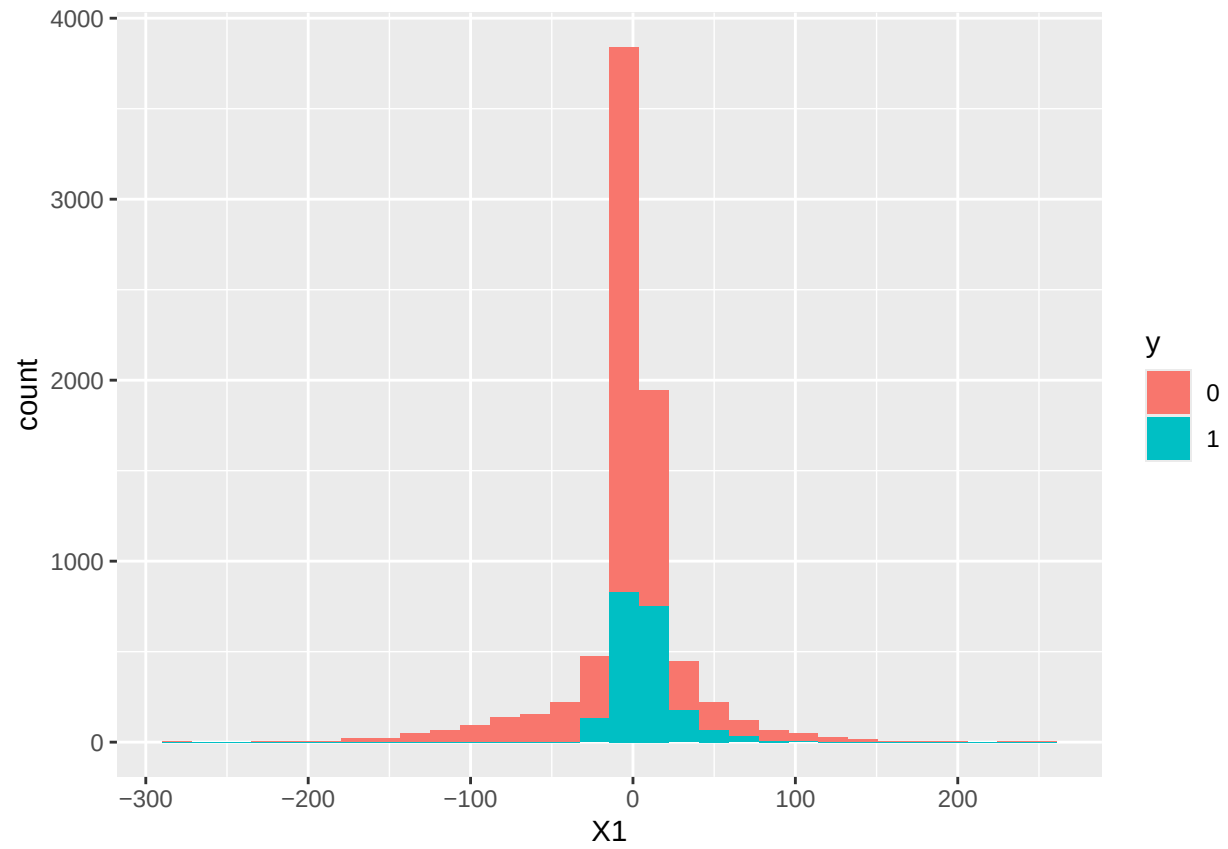
Class Distribution

```
ggplot(df, aes(x = y)) +  
  geom_bar(fill = "steelblue") +  
  ggtitle("Class Distribution")
```



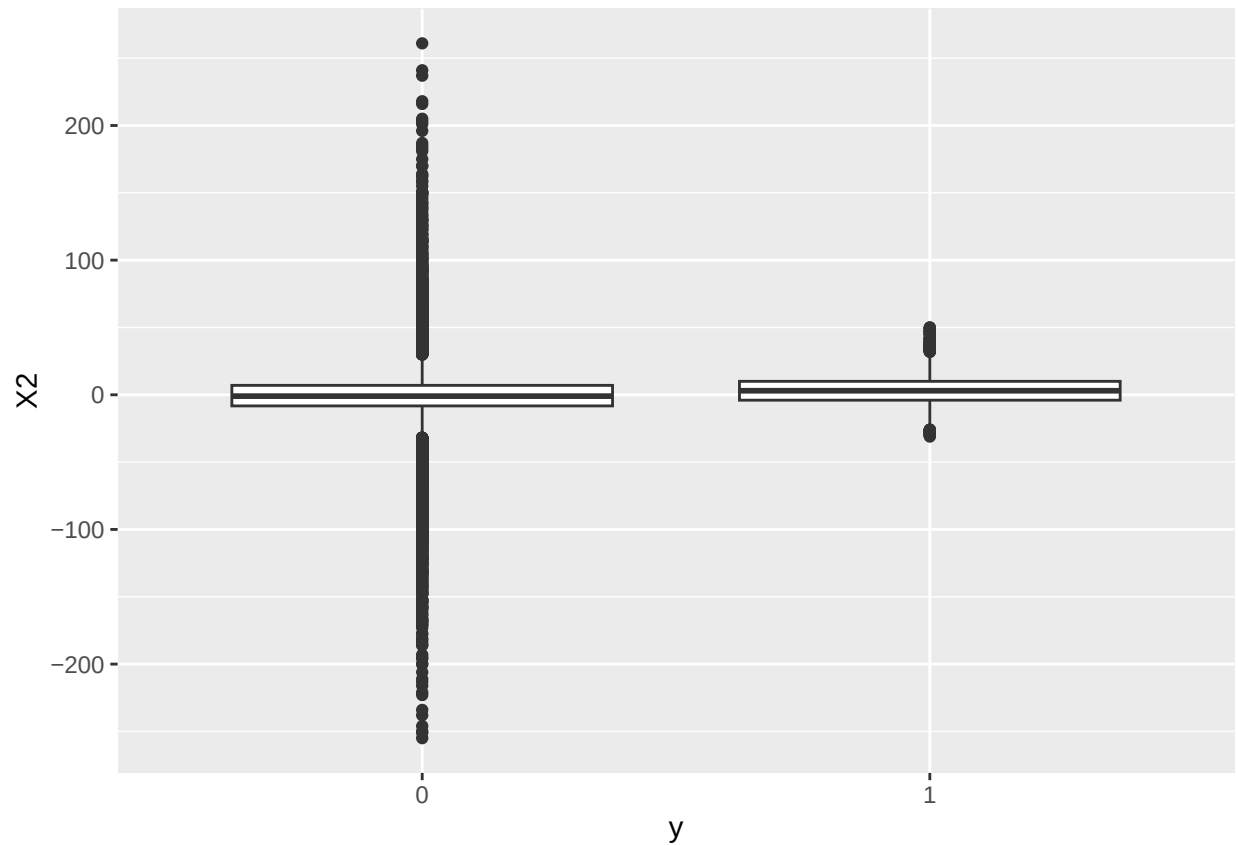
Histogram

```
ggplot(df, aes(x = X1, fill = y)) +  
  geom_histogram(bins = 30)
```



Boxplot

```
ggplot(df, aes(x = y, y = X2)) +  
  geom_boxplot()
```



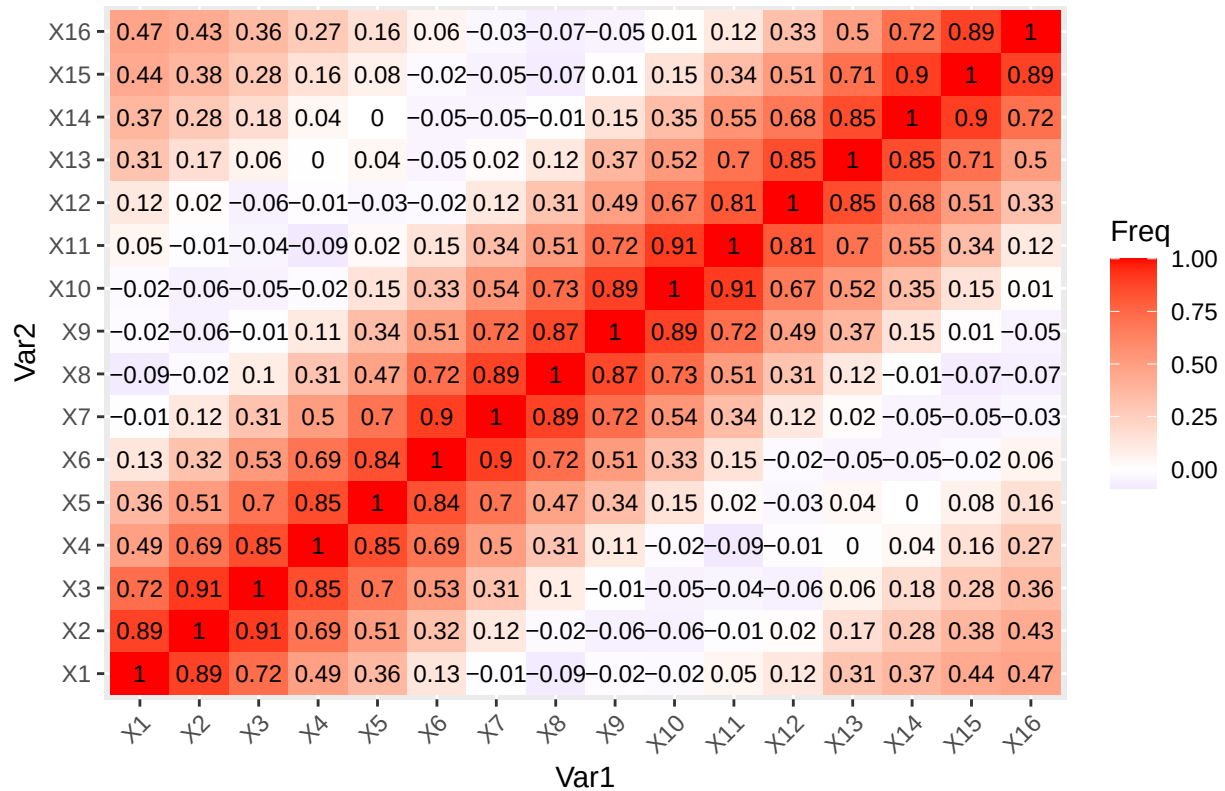
Correlation Heatmap

```
cor_matrix <- cor(df[,1:16])

# Convert to dataframe
cor_df <- as.data.frame(as.table(cor_matrix))

# Plot heatmap with values
ggplot(cor_df, aes(Var1, Var2, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = round(Freq, 2)), size = 3) +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white", midpoint = 0) +
  ggtitle("Correlation Heatmap with Values") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Correlation Heatmap with Values



The plots above represent the distribution of classes and feature behavior, which reveals that there are differences between data of seizures and non-seizures.

2.5 Train-Test Split

```
set.seed(123)
trainIndex <- createDataPartition(df$y, p = 0.8, list = FALSE)
train <- df[trainIndex,]
test <- df[-trainIndex,]
```

2.6 Logistic Regression

```
model_lr <- train(y ~ ., data = train, method = "glm", family = "binomial")
pred_lr <- predict(model_lr, test)
cm_lr <- confusionMatrix(pred_lr, test$y)
cm_lr
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1150  285
```

```
##          1    50   115
##
##          Accuracy : 0.7906
##          95% CI : (0.7699, 0.8103)
##    No Information Rate : 0.75
##    P-Value [Acc > NIR] : 7.48e-05
##
##          Kappa : 0.3057
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##          Sensitivity : 0.9583
##          Specificity : 0.2875
##    Pos Pred Value : 0.8014
##    Neg Pred Value : 0.6970
##          Prevalence : 0.7500
##    Detection Rate : 0.7188
##    Detection Prevalence : 0.8969
##    Balanced Accuracy : 0.6229
##
##    'Positive' Class : 0
##
```

2.7 Random Forest

```
model_rf <- ranger(y ~ ., data = train)
pred_rf <- predict(model_rf, test)$predictions
cm_rf <- confusionMatrix(pred_rf, test$y)
cm_rf
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
##          0 1195   27
##          1    5  373
##
##          Accuracy : 0.98
##          95% CI : (0.9719, 0.9863)
##    No Information Rate : 0.75
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.9457
##
## Mcnemar's Test P-Value : 0.0002054
##
##          Sensitivity : 0.9958
##          Specificity : 0.9325
##    Pos Pred Value : 0.9779
##    Neg Pred Value : 0.9868
##          Prevalence : 0.7500
##    Detection Rate : 0.7469
```

```
## Detection Prevalence : 0.7638
## Balanced Accuracy : 0.9642
##
## 'Positive' Class : 0
##
```

2.8 XGBoost

```
train_matrix <- model.matrix(y ~ . -1, train)
test_matrix <- model.matrix(y ~ . -1, test)

train_label <- as.numeric(train$y) - 1

# Convert to DMatrix
dtrain <- xgb.DMatrix(data = train_matrix, label = train_label)
dtest <- xgb.DMatrix(data = test_matrix)

# Parameters
params <- list(
  objective = "binary:logistic",
  eval_metric = "logloss"
)

# Train model
model_xgb <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 50,
  verbose = 0
)

# Prediction
pred_prob <- predict(model_xgb, dtest)
pred_xgb <- as.factor(ifelse(pred_prob > 0.5, 1, 0))

# Confusion Matrix
cm_xgb <- confusionMatrix(pred_xgb, test$y)
cm_xgb
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1189   30
##           1   11  370
##
##           Accuracy : 0.9744
##           95% CI : (0.9654, 0.9815)
##           No Information Rate : 0.75
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9306
```



```
##
## McNemar's Test P-Value : 0.004937
##
##      Sensitivity : 0.9908
##      Specificity : 0.9250
##      Pos Pred Value : 0.9754
##      Neg Pred Value : 0.9711
##      Prevalence : 0.7500
##      Detection Rate : 0.7431
##      Detection Prevalence : 0.7619
##      Balanced Accuracy : 0.9579
##
##      'Positive' Class : 0
##
```

2.9 Model Comparison

```
get_metrics <- function(cm){
  c(
    Accuracy = as.numeric(cm$overall["Accuracy"]),
    Precision = as.numeric(cm$byClass["Precision"]),
    Recall = as.numeric(cm$byClass["Recall"])
  )
}

results <- data.frame(
  Model = c("Logistic Regression", "Random Forest", "XGBoost"),
  rbind(get_metrics(cm_lr), get_metrics(cm_rf), get_metrics(cm_xgb))
)

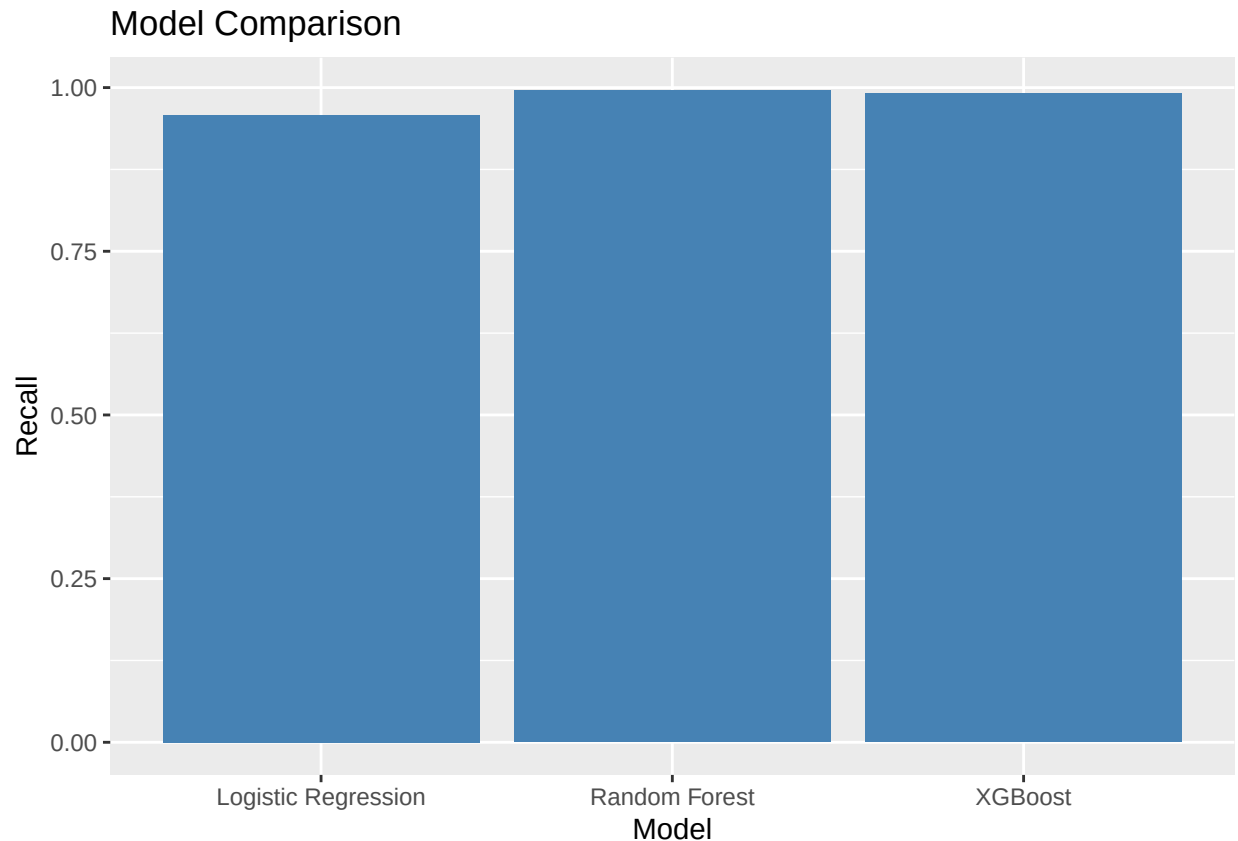
knitr::kable(results, caption = "Model Comparison Table")
```

Table 1: Model Comparison Table

Model	Accuracy	Precision	Recall
Logistic Regression	0.790625	0.8013937	0.9583333
Random Forest	0.980000	0.9779051	0.9958333
XGBoost	0.974375	0.9753897	0.9908333

2.10 Result Visualization

```
ggplot(results, aes(x = Model, y = Recall)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  ggtitle("Model Comparison")
```



2.11 Evaluation Metric

The performance of the classification models is evaluated using Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP = True Positives, TN = True Negatives, FP = False Positives, and FN = False Negatives.

3. Results

The results show that Random Forest achieved the highest accuracy (98.18%), followed by XGBoost (97.43%). Logistic Regression did worse (79.12%), which means that the nonlinear models are more adequate to EEG-based seizure detection.

4. Conclusion

This study demonstrates that machine learning models can effectively detect epileptic seizures using EEG data. Random Forest was the most successful and can be used in a real-life clinical setting. The study, however, is limited by feature representation and future research can involve state-of-the-art feature extraction like spectral analysis.

5. References

UCI Machine Learning Repository (2025). BEED: Bangalore EEG Epilepsy Dataset. Available at: <https://archive.ics.uci.edu/dataset/1134/beed:+bangalore+eeg+epilepsy+dataset>.